

MERN Stack Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

1. Programming & Web Development Fundamentals

- Introduction to software development
- What is full-stack development?
- What is MERN Stack?
- MERN Stack architecture
- Frontend vs backend development
- Client-server architecture
- Three-tier architecture
- Single Page Applications
- How web applications work
- Browser and server responsibilities
- HTTP and HTTPS
- Request-response lifecycle
- REST APIs
- JSON
- Development, testing, staging, and production environments
- IDE setup
- Visual Studio Code
- Browser developer tools
- Terminal and command-line fundamentals
- Introduction to Git and GitHub
- Introduction to npm
- Introduction to Node.js ecosystem

Part I — Web Fundamentals

2. HTML5

- Introduction to HTML
- HTML document structure
- Elements
- Tags
- Attributes
- Headings
- Paragraphs
- Text formatting
- Hyperlinks
- Images
- Lists
- Tables
- Forms
- Input elements
- Buttons
- Select controls
- Radio buttons
- Checkboxes
- File inputs

- Semantic HTML
- Header
- Footer
- Nav
- Main
- Section
- Article
- Aside
- Audio
- Video
- iframe
- Meta tags
- Favicons
- HTML entities
- Data attributes
- Accessibility fundamentals
- SEO-friendly HTML

3. CSS3

- Introduction to CSS
- CSS syntax
- Inline CSS
- Internal CSS
- External CSS
- CSS selectors
- Element selector
- Class selector
- ID selector
- Attribute selector
- Combinators
- Colors
- Backgrounds
- Typography
- Web fonts
- CSS units
- px
- %
- em
- rem
- vh
- vw

CSS Box Model

- Content
- Padding
- Border
- Margin
- Box sizing

Layout & Positioning

- Display
- Position
- Relative

- Absolute
- Fixed
- Sticky
- Z-index
- Overflow

4. Modern CSS Layout

Flexbox

- Flex container
- Flex items
- Flex direction
- Justify content
- Align items
- Align content
- Flex wrap
- Gap
- Responsive flex layouts

CSS Grid

- Grid container
- Rows
- Columns
- Grid areas
- Gap
- Responsive grids

Responsive Web Design

- Mobile-first development
- Media queries
- Breakpoints
- Responsive units
- Responsive images
- Adaptive layouts

5. Advanced CSS

- Pseudo-classes
- Pseudo-elements
- CSS variables
- Transitions
- Transformations
- Animations
- Keyframes
- Shadows
- Gradients
- Responsive typography
- CSS architecture basics
- Reusable utility classes

6. Bootstrap / Tailwind CSS

Students can learn one or both.

Bootstrap

- Containers
- Grid system
- Breakpoints
- Buttons
- Forms
- Tables
- Cards
- Alerts
- Navbar
- Dropdowns
- Modals
- Offcanvas
- Responsive utilities

Tailwind CSS

- Utility-first CSS
- Layout utilities
- Flexbox
- Grid
- Spacing
- Typography
- Responsive utilities
- State variants
- Reusable component patterns
- Custom configuration

Part II — JavaScript Programming

7. JavaScript Fundamentals

- Introduction to JavaScript
- JavaScript engines
- Adding JavaScript to HTML
- Statements
- Comments
- Variables
- var
- let
- const
- Identifiers
- Data types
- Primitive types
- Reference types
- typeof
- Type conversion
- Type coercion

Operators

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Bitwise operators

- Conditional operator
- Nullish coalescing

8. JavaScript Control Flow

Conditional Statements

- if
- if-else
- else-if
- Nested conditions
- switch

Loops

- for
- while
- do-while
- for...of
- for...in
- Nested loops
- break
- continue

9. JavaScript Functions

- Function declarations
- Function expressions
- Parameters
- Arguments
- Return values
- Default parameters
- Anonymous functions
- Arrow functions
- Callback functions
- Higher-order functions
- Rest parameters
- Spread operator
- Recursion
- Closures
- Lexical scope
- Function scope
- Block scope
- Hoisting
- IIFE concepts

10. Arrays & Objects

Arrays

- Creating arrays
- Accessing elements
- Updating elements
- Array methods
- push
- pop
- shift

- unshift
- slice
- splice
- concat
- includes
- indexOf

Functional Array Methods

- map
- filter
- reduce
- find
- findIndex
- some
- every
- sort
- forEach

Objects

- Creating objects
- Properties
- Methods
- Nested objects
- Object iteration
- Object destructuring
- Object spread
- Object methods
- Optional chaining

11. Strings, Dates & Built-In Objects

- Strings
- String methods
- Template literals
- Regular expressions
- Math object
- Date object
- JSON
- `JSON.stringify()`
- `JSON.parse()`
- Set
- Map
- WeakSet concepts
- WeakMap concepts

12. Object-Oriented JavaScript

- Objects
- Prototypes
- Prototype chain
- Constructor functions
- Classes
- Constructors
- Properties

- Methods
- Encapsulation
- Inheritance
- Polymorphism
- Static members
- Private fields
- Getters
- Setters
- `this`
- `call`
- `apply`
- `bind`

13. Modern JavaScript — ES6+

- `let`
- `const`
- Arrow functions
- Template literals
- Destructuring
- Spread operator
- Rest operator
- Default parameters
- Modules
- `import`
- `export`
- Classes
- Promises
- Async/await
- Optional chaining
- Nullish coalescing
- Enhanced object literals

Part III — Browser & DOM Programming

14. DOM Manipulation

- Document Object Model
- DOM tree
- Selecting elements
- `getElementById`
- `querySelector`
- `querySelectorAll`
- Changing content
- Changing attributes
- Changing styles
- Creating elements
- Removing elements
- DOM traversal
- Classes
- `classList`

15. Events

- Event-driven programming
- Click events
- Keyboard events
- Mouse events
- Form events
- Input events
- Event listeners
- Event object
- Event bubbling
- Event capturing
- Event delegation
- Preventing default behavior

16. Forms & Validation

- Form controls
- Reading form data
- Client-side validation
- Required fields
- Pattern validation
- Custom validation
- Error messages
- Form submission
- Preventing invalid submissions

17. Browser Storage

- Cookies
- Local Storage
- Session Storage
- Storage APIs
- JSON storage
- Authentication storage considerations
- Security considerations

Part IV — Asynchronous JavaScript

18. Asynchronous Programming

- Synchronous vs asynchronous execution
- JavaScript runtime
- Call stack
- Web APIs
- Callback queue
- Event loop
- Callbacks
- Callback hell
- Promises
- Promise states
- Promise chaining
- Error handling
- `async`
- `await`

- try/catch
- Parallel asynchronous operations
- Promise.all
- Promise.allSettled

19. AJAX & API Communication

- AJAX fundamentals
- HTTP methods
- Fetch API
- GET
- POST
- PUT
- PATCH
- DELETE
- Request headers
- Request bodies
- Response objects
- JSON
- API error handling
- Loading states
- Consuming third-party APIs

Part V — TypeScript for MERN Developers

Although MERN can be developed entirely in JavaScript, modern production projects benefit greatly from TypeScript.

20. TypeScript Fundamentals

- JavaScript vs TypeScript
- TypeScript installation
- TypeScript compiler
- tsconfig.json
- Static typing
- Type inference
- Primitive types
- Arrays
- Tuples
- Enums
- any
- unknown
- never
- void
- Union types
- Intersection types
- Literal types

21. TypeScript Functions & Objects

- Typed parameters
- Return types
- Optional parameters
- Default parameters
- Function types

- Object types
- Type aliases
- Interfaces
- Readonly properties
- Optional properties

22. TypeScript OOP

- Classes
- Constructors
- Access modifiers
- Public
- Private
- Protected
- Properties
- Methods
- Inheritance
- Abstract classes
- Interfaces
- Static members
- Generics

23. Advanced TypeScript

- Generic types
- Generic constraints
- Utility types
- Partial
- Pick
- Omit
- Record
- keyof
- typeof
- Type guards
- Discriminated unions
- Modules
- Decorator concepts

Part VI — React Frontend Development

24. React Fundamentals

- Introduction to React
- Why React?
- SPA architecture
- React ecosystem
- Project creation
- Vite
- Project structure
- JSX
- Components
- Functional components
- Component composition

- Reusable components
- Props
- State
- Events
- Conditional rendering
- Lists
- Keys

25. React Components

- Creating components
- Functional components
- Parent-child relationships
- Component composition
- Passing props
- Default props concepts
- Children
- Reusable UI components
- Smart vs presentational component concepts

26. React State & Events

- State concepts
- useState
- Updating state
- Immutable state
- Event handling
- Form events
- Passing event handlers
- Lifting state up
- Sharing state between components

27. React Hooks

- Rules of hooks
- useState
- useEffect
- useRef
- useContext
- useReducer
- useMemo
- useCallback
- Custom hooks
- Reusable hook logic
- Managing side effects

28. React Lifecycle Concepts

- Component mounting
- Updating
- Unmounting
- Effects
- Cleanup functions
- Dependency arrays
- Avoiding unnecessary effects

29. React Forms

- Controlled components
- Uncontrolled components
- Form state
- Form validation
- Error messages
- Multi-field forms
- Dynamic forms
- File uploads
- React Hook Form
- Schema validation concepts

Possible validation tools:

- Zod
- Yup

30. React Router

- Client-side routing
- React Router
- BrowserRouter
- Routes
- Route
- Link
- NavLink
- Route parameters
- Query parameters
- Nested routes
- Layout routes
- Programmatic navigation
- Protected routes
- Authentication routes
- 404 pages

31. React Context API

- Context
- Provider
- Consumer concepts
- useContext
- Global state
- Authentication context
- Theme context
- Avoiding prop drilling

32. React State Management

Redux Toolkit

- Why global state?
- Redux fundamentals
- Store
- Slices
- Reducers

- Actions
- Selectors
- `useSelector`
- `useDispatch`
- Async logic
- `createAsyncThunk`
- Redux DevTools

Alternative / additional option:

- Zustand

33. React API Integration

- Fetch API
- Axios
- GET requests
- POST requests
- PUT
- PATCH
- DELETE
- Environment variables
- API base URLs
- Reusable API services
- Loading states
- Error handling
- Authentication headers
- Axios interceptors
- Token refresh concepts

34. Server State & Data Fetching

Introduction to modern data-fetching approaches:

- Server state vs client state
- Caching API responses
- Loading and stale data
- Background refetching
- Pagination
- Infinite scrolling

Optional tools:

- TanStack Query / React Query

35. React Performance Optimization

- Rendering behavior
- Re-rendering
- `React.memo`
- `useMemo`
- `useCallback`
- Code splitting
- Lazy loading
- Suspense concepts
- Bundle optimization
- Image optimization

- Avoiding unnecessary renders

36. React UI Development

- Reusable layouts
- Navbar
- Sidebar
- Cards
- Tables
- Forms
- Modals
- Dialogs
- Alerts
- Toast notifications
- Pagination
- Search
- Filtering
- Sorting
- Loading spinners
- Skeleton loaders
- Responsive UI design

Optional UI libraries:

- Material UI
- Ant Design
- Chakra UI
- shadcn/ui concepts
- Tailwind CSS

Part VII — Node.js Backend Development

37. Node.js Fundamentals

- Introduction to Node.js
- Why Node.js?
- Node.js architecture
- V8 engine
- Event-driven architecture
- Non-blocking I/O
- Event loop
- Node.js installation
- REPL
- Creating Node.js applications
- Running scripts

38. Node.js Modules

- CommonJS
- ES Modules
- `require`
- `module.exports`
- `import`
- `export`
- Built-in modules

- Custom modules
- Third-party modules

39. npm & Package Management

- npm
- Package registry
- `package.json`
- Dependencies
- Dev dependencies
- Installing packages
- Updating packages
- Semantic versioning
- `package-lock.json`
- npm scripts
- npm audit
- Dependency management

40. Node.js Core Modules

- `fs`
- `path`
- `http`
- `https`
- `url`
- `events`
- `os`
- `crypto`
- Streams
- Buffers

41. Node.js Event Loop

- Event loop architecture
- Call stack
- Callback queue
- Microtasks
- Timers
- EventEmitter
- Async I/O
- Blocking vs non-blocking code
- CPU-bound vs I/O-bound operations

42. File Handling

- Reading files
- Writing files
- Appending files
- Deleting files
- Working with directories
- Synchronous operations
- Asynchronous operations
- Streams
- Buffers
- File upload concepts

Part VIII — Express.js Backend Development

43. Express.js Fundamentals

- Introduction to Express
- Installing Express
- Creating an Express application
- Express server
- Request object
- Response object
- Routing
- Route parameters
- Query parameters
- Request body
- JSON parsing

44. Express Routing

- GET
- POST
- PUT
- PATCH
- DELETE
- Router
- Modular routing
- Route parameters
- Query parameters
- Nested routes
- RESTful routing conventions

45. Express Middleware

- Middleware fundamentals
- Application-level middleware
- Router-level middleware
- Built-in middleware
- Third-party middleware
- Custom middleware
- Authentication middleware
- Authorization middleware
- Logging middleware
- Error-handling middleware

46. REST API Development

- REST architecture
- Resource-based APIs
- HTTP methods
- HTTP status codes
- Request headers
- Request bodies
- JSON responses
- CRUD APIs
- API validation

- Pagination
- Filtering
- Sorting
- Search
- API versioning
- Error responses

47. Backend Project Architecture

Recommended architecture:

Routes → Controllers → Services → Models → Database

Topics:

- Separation of concerns
- Route layer
- Controller layer
- Service layer
- Repository/data-access concepts
- Models
- Utilities
- Middleware
- Configuration
- Centralized error handling
- Validation
- Logging

Part IX — MongoDB

48. MongoDB Fundamentals

- SQL vs NoSQL
- MongoDB architecture
- Documents
- Collections
- Databases
- BSON
- MongoDB installation
- MongoDB Compass
- MongoDB Atlas

49. MongoDB CRUD Operations

- Insert one
- Insert many
- Find
- Find one
- Update one
- Update many
- Delete one
- Delete many
- Replace operations concepts

50. MongoDB Queries

- Query operators
- Comparison operators
- Logical operators
- Array operators
- Projection
- Sorting
- Limit
- Skip
- Pagination
- Regular expression searches

51. MongoDB Data Modeling

- Document modeling
- Embedded documents
- Referenced documents
- One-to-one
- One-to-many
- Many-to-many
- Denormalization
- Modeling for access patterns
- Schema design considerations

52. MongoDB Aggregation Framework

- Aggregation pipeline
- \$match
- \$project
- \$group
- \$sort
- \$limit
- \$skip
- \$unwind
- \$lookup
- \$count
- Aggregation expressions
- Report generation

53. MongoDB Indexing & Performance

- Indexes
- Single-field indexes
- Compound indexes
- Unique indexes
- Text indexes
- Query optimization
- Explain plans
- Index selection
- Performance considerations

Part X — Mongoose ODM

54. Mongoose Fundamentals

- ODM concepts
- Connecting Node.js with MongoDB
- Schemas
- Models
- Documents
- Data types
- Validation
- Required fields
- Defaults
- Timestamps

55. Mongoose CRUD Operations

- Create
- Save
- Find
- FindById
- FindOne
- Update
- Delete
- Query chaining
- Filtering
- Sorting
- Pagination

56. Mongoose Relationships

- ObjectId
- References
- Population
- Embedded schemas
- Nested documents
- One-to-many modeling
- Many-to-many modeling
- Virtual fields

57. Advanced Mongoose

- Instance methods
- Static methods
- Pre middleware
- Post middleware
- Custom validators
- Transactions
- Aggregation
- Indexes
- Query optimization
- Lean queries concepts

Part XI — Authentication & Authorization

58. User Authentication

- Authentication vs authorization

- Registration
- Login
- Logout
- Password hashing
- bcrypt
- Password validation
- User profiles
- Forgot-password concepts
- Password-reset workflows
- Email verification concepts

59. JWT Authentication

- What is JWT?
- JWT structure
- Header
- Payload
- Signature
- Access token
- Refresh token
- Token generation
- Token verification
- Expiration
- Authentication middleware
- Protected routes
- React authentication integration

60. Role-Based Authorization

- Roles
- Permissions
- Admin
- User
- Manager roles
- Resource ownership
- Backend middleware
- Protected React routes
- Role-based UI rendering

61. OAuth & Social Authentication

- OAuth 2.0 concepts
- OpenID Connect concepts
- Social authentication
- Google sign-in concepts
- Microsoft sign-in concepts
- GitHub authentication concepts
- Single Sign-On introduction

Part XII — MERN Stack Integration

62. Complete MERN Architecture

Students should understand:

React → Express.js → Node.js → MongoDB

Detailed flow:

React Component → API Service → Express Route → Controller → Service → Mongoose Model → MongoDB

63. React + Node.js Integration

- Connecting React to Express API
- Fetch/Axios
- CORS
- Environment variables
- GET
- POST
- PUT
- PATCH
- DELETE
- Loading states
- Error handling
- Form submission
- File upload
- Search
- Filtering
- Sorting
- Pagination

64. Full-Stack Authentication

Complete flow:

React Login → Express API → MongoDB User Verification → JWT → React Auth State → Protected Route

Topics:

- User registration
- User login
- Auth context
- JWT handling
- Refresh tokens concepts
- Axios interceptors
- Protected routes
- Backend authorization
- Role-based UI

Part XIII — REST API Best Practices

65. REST API Design

- Resource naming
- REST conventions
- HTTP methods
- Status codes
- Request/response structures
- Validation
- Error handling
- Pagination

- Filtering
- Sorting
- Searching
- Versioning
- Idempotency
- Rate limiting

66. API Validation

Possible tools:

- Zod
- Joi
- express-validator

Topics:

- Body validation
- Query validation
- Route parameter validation
- Custom validation
- Sanitization
- Validation middleware
- Centralized errors

67. Swagger & OpenAPI

- API documentation
- OpenAPI specification
- Swagger UI
- Endpoint documentation
- Request schemas
- Response schemas
- Authentication documentation
- Testing APIs using Swagger

Part XIV — Application Security

68. Web Application Security

- Authentication
- Authorization
- Password hashing
- HTTPS
- TLS
- CORS
- CSRF
- XSS
- NoSQL injection
- Command injection
- Input validation
- Output escaping
- Secure headers
- Helmet
- Rate limiting

- Brute-force protection
- Cookie security
- JWT security
- Secrets management
- Environment variables
- OWASP Top 10 awareness

69. Secure MERN Development

- Secure React applications
- Avoiding exposed secrets
- Secure backend validation
- Password hashing
- Secure cookies
- HttpOnly cookies
- SameSite
- Token security
- Environment-based configuration
- Secure HTTP headers
- API request limits
- Dependency vulnerability checks

Part XV — File Upload & Media Handling

70. File Uploads

- Multipart/form-data
- Multer
- File upload API
- Upload validation
- File size limits
- MIME type validation
- Images
- Documents
- Profile pictures
- Product images

71. Cloud File Storage Concepts

Introduction to:

- AWS S3 concepts
- Azure Blob Storage concepts
- Cloudinary concepts
- Signed URLs
- Public vs private assets
- CDN fundamentals

Part XVI — Real-Time Applications

72. WebSockets

- HTTP vs WebSockets

- Persistent connections
- Real-time communication
- Client-server messaging
- Connection lifecycle

73. Socket.IO

- Socket.IO server
- Socket.IO client
- Events
- Broadcasting
- Rooms
- Namespaces concepts
- Online presence
- Real-time notifications
- Chat systems
- React integration

Part XVII — Advanced Node.js

74. Node.js Performance

- Event loop performance
- Async I/O
- Streams
- Buffers
- Blocking operations
- CPU-intensive tasks
- Worker Threads concepts
- Clustering concepts
- Memory management
- Performance monitoring

75. Redis & Caching

- Introduction to Redis
- Key-value databases
- In-memory caching
- API response caching
- Session storage
- Cache expiration
- Cache invalidation
- Distributed caching concepts
- Rate limiting
- Performance improvement

76. Background Jobs

- Background processing
- Job queues
- Scheduled jobs
- Cron jobs
- Email jobs
- Report generation

- Image processing concepts
- BullMQ concepts
- Redis-backed queues
- Retry mechanisms
- Failed jobs

77. Email & Notifications

- Sending emails
- Nodemailer concepts
- HTML emails
- Email verification
- Password-reset emails
- Order confirmation emails
- Notification services
- Push-notification concepts

Part XVIII — Testing

78. JavaScript Testing Fundamentals

- Why testing matters
- Unit testing
- Integration testing
- End-to-end testing
- Test cases
- Test suites
- Assertions
- Test-driven development concepts

Possible tools:

- Jest
- Vitest

79. React Testing

- React Testing Library
- Component testing
- Rendering components
- Querying elements
- User interactions
- Form testing
- Hooks testing concepts
- API mocking
- Async component testing

80. Node.js & Express Testing

- Unit testing services
- Controller testing
- Route testing
- Middleware testing
- API integration testing
- Supertest

- Authentication tests
- Error-response testing

81. API Testing

- Postman
- Collections
- Environments
- Variables
- Authorization
- Request chaining
- Test scripts
- Swagger testing
- Automated API testing concepts

Part XIX — Git & GitHub

82. Git Fundamentals

- Version control
- Git installation
- Repository creation
- `git init`
- `git clone`
- `git status`
- `git add`
- `git commit`
- `git push`
- `git pull`
- `git fetch`

83. Git Collaboration

- Branches
- Feature branches
- Merging
- Rebase concepts
- Merge conflicts
- Pull requests
- Code reviews
- GitHub Issues
- Tags
- Releases
- `.gitignore`
- README files
- Branching strategies

Part XX — Software Engineering Practices

84. Clean Code

- Naming conventions
- Modularity

- Readability
- DRY
- KISS
- YAGNI
- Separation of concerns
- Reusable functions
- Reusable components
- Error handling
- Logging
- Documentation
- Code reviews

85. SOLID Principles

- Single Responsibility Principle
- Open/Closed Principle
- Liskov Substitution Principle
- Interface Segregation Principle
- Dependency Inversion Principle

Applied using JavaScript/TypeScript and Node.js.

86. Design Patterns

Introduction to:

- Module Pattern
- Factory Pattern
- Singleton Pattern
- Repository Pattern
- Service Pattern
- Strategy Pattern
- Observer Pattern
- Dependency Injection
- MVC
- Middleware Pattern

87. Agile Development

- Agile methodology
- Scrum
- Sprint
- Product backlog
- User stories
- Acceptance criteria
- Sprint planning
- Daily stand-up
- Retrospective
- Code review
- Jira fundamentals
- Team collaboration

Part XXI — Deployment & DevOps

88. Linux Fundamentals

- Basic commands
- Directory structure
- Files
- File permissions
- Processes
- Package management
- Environment variables
- Networking basics
- SSH
- Logs
- Running Node.js applications

89. Deployment Environments

Students should understand:

Development → Testing → Staging → Production

Topics:

- Environment-specific configuration
- .env files
- Secrets
- Frontend builds
- Backend configuration
- Database connections
- Logging
- Monitoring
- Production best practices

90. Production Node.js Deployment

- Running Node.js in production
- PM2 concepts
- Reverse proxies
- Nginx concepts
- HTTPS
- SSL certificates
- Domains
- DNS
- Environment variables
- Logging
- Process restart strategies

91. Docker

- Containers
- Containers vs virtual machines
- Docker installation
- Docker images
- Docker containers
- Dockerfile
- Ports
- Volumes

- Networks
- Environment variables
- .dockerignore

Students containerize:

- React frontend
- Node.js / Express backend
- MongoDB

Architecture:

React Container → Node/Express Container → MongoDB Container

92. Docker Compose

- Multi-container applications
- Docker Compose configuration
- Services
- Volumes
- Networks
- Environment variables
- Starting/stopping the stack
- Container communication

93. CI/CD

- Continuous Integration
- Continuous Delivery
- Continuous Deployment
- Build pipelines
- Automated testing
- Deployment pipelines

Tools:

- GitHub Actions
- Azure DevOps concepts
- GitLab CI concepts

Typical flow:

Commit → Install → Lint → Test → Build → Docker Image → Deploy

Part XXII — Cloud Deployment

94. Cloud Fundamentals

- Cloud computing
- IaaS
- PaaS
- SaaS
- Regions
- Availability
- Virtual machines
- Application hosting
- Object storage
- Managed databases

- Domains
- DNS
- SSL/TLS

95. Deploying MERN Applications

Platforms may include:

- AWS
- Microsoft Azure
- Google Cloud
- Render
- Railway
- Vercel
- Netlify
- MongoDB Atlas

Students should deploy:

- React frontend
- Node.js backend
- MongoDB database
- Environment configuration
- Domain
- HTTPS

Part XXIII — Monitoring & Observability

96. Application Logging

- Logging concepts
- Log levels
- Error logs
- Access logs
- Structured logging
- Request logging
- Winston concepts
- Morgan concepts
- Correlation IDs

97. Monitoring Fundamentals

- Health checks
- Metrics
- Logs
- Traces
- Error monitoring
- Performance monitoring
- Availability monitoring
- Production troubleshooting
- Observability fundamentals

Part XXIV — Microservices Fundamentals

98. Monolith vs Microservices

- Monolithic architecture
- Modular monolith
- Microservices
- Service decomposition
- Advantages/disadvantages
- Independent deployment
- Database-per-service concepts
- Synchronous communication
- Asynchronous communication

99. Node.js Microservices

- Independent services
- REST communication
- Service-to-service communication
- API Gateway concepts
- Authentication between services
- Message queues
- Redis
- RabbitMQ concepts
- Kafka concepts
- Event-driven architecture
- Retry patterns
- Circuit breaker concepts

Part XXV — Data Structures & Algorithms

100. DSA with JavaScript

Data Structures

- Arrays
- Strings
- Stack
- Queue
- Linked list
- Hash maps
- Sets
- Trees
- Binary trees
- Binary search trees
- Heap
- Graphs

Algorithms

- Linear search
- Binary search
- Bubble sort
- Selection sort
- Insertion sort
- Merge sort
- Quick sort

- Recursion
- Hashing
- Two-pointer technique
- Sliding window
- Basic dynamic programming

Complexity

- Time complexity
- Space complexity
- Big-O notation

Part XXVI — AI-Assisted MERN Development

101. AI-Assisted Software Development

Tools can include:

- GitHub Copilot
- ChatGPT
- Claude
- Cursor
- AI coding agents

Topics:

- Prompting for code
- Generating React components
- Generating Express APIs
- MongoDB query generation
- JavaScript/TypeScript code generation
- Debugging with AI
- Refactoring
- Test generation
- API test generation
- Documentation generation
- Code-review assistance
- Explaining unfamiliar code
- AI-assisted database modeling
- AI-assisted API design
- Git assistance
- Context engineering
- Coding agents
- Validating AI-generated code
- Security risks
- Avoiding blind copy-paste development

Part XXVII — Generative AI Integration

102. Generative AI Fundamentals

- Introduction to Generative AI
- Large Language Models
- Prompt engineering
- System prompts

- User prompts
- Tokens
- Context windows
- LLM APIs
- Structured output
- JSON responses

103. Integrating LLMs with Node.js

- Calling LLM APIs
- Server-side integration
- Protecting API keys
- Prompt templates
- Chat history
- Streaming responses
- Structured outputs
- Error handling
- Rate limiting
- Function/tool calling concepts

Architecture:

React → Node.js/Express → LLM API

104. RAG Fundamentals

- Retrieval-Augmented Generation
- Embeddings
- Vector representations
- Vector databases
- Chunking
- Retrieval
- Semantic search
- Context injection
- Prompt augmentation
- Grounded responses

Architecture:

React → Express → LLM + Vector Database → Knowledge Base

105. AI-Powered MERN Applications

Students can build:

- AI chatbot
- AI customer-support application
- Document Q&A system
- Resume analyzer
- AI content generator
- AI-powered search
- AI-based learning platform
- FAQ assistant
- Product-description generator
- AI-powered CRM assistant

Part XXVIII — Projects

106. JavaScript Mini Projects

Examples:

- Calculator
- Todo application
- Quiz application
- Expense tracker
- Notes application
- Weather application
- Form-validation application

107. React Mini Projects

- Personal portfolio
- Admin dashboard
- Task management UI
- Product catalog
- E-commerce frontend
- Weather dashboard
- Notes application
- API-based application

108. Node.js / Express API Projects

- Student Management API
- Employee Management API
- Product Management API
- Authentication API
- Blog API
- Task Management API
- E-commerce backend

109. MongoDB Projects

- Student database
- Employee database
- Product catalog
- Blogging database
- E-commerce database
- Social media data model

110. Full-Stack Project 1 — Task Management System

Technology:

React + Node.js + Express.js + MongoDB

Features:

- Registration
- Login
- JWT authentication
- Dashboard
- Task creation

- Task editing
- Categories
- Priorities
- Status
- Search
- Filtering
- Pagination
- Role-based authorization

111. Full-Stack Project 2 — E-Commerce Application

Features:

- User registration
- Login
- Product catalog
- Categories
- Product search
- Filtering
- Product detail pages
- Shopping cart
- Wishlist
- Orders
- Product images
- Admin dashboard
- Role management
- Payment gateway concepts
- REST APIs

112. Full-Stack Project 3 — Job Portal

Features:

- Candidate registration
- Employer registration
- Authentication
- Job posting
- Job search
- Job applications
- Resume upload
- Candidate dashboard
- Employer dashboard
- Role-based access
- Notifications

Part XXIX — Capstone Project

113. Industry-Style Capstone Project

Students should complete the complete software development lifecycle:

Requirement Analysis → UI Design → Database Design → React Frontend → Node/Express Backend → MongoDB → REST APIs → Authentication → Testing → Docker → CI/CD → Cloud Deployment

Possible projects:

- Learning Management System
- CRM
- Hospital Management System
- Recruitment Portal
- E-commerce platform
- Food-delivery application
- Project management system
- Hotel booking platform
- Social networking application
- Inventory management system
- SaaS application
- AI-powered customer-support platform

Part XXX — GitHub & Portfolio Development

114. GitHub Profile Building

- Professional GitHub profile
- Profile README
- Repository organization
- Repository naming
- Meaningful commits
- README documentation
- Project screenshots
- Architecture diagrams
- API documentation
- Installation instructions
- Demo links
- Deployment links
- GitHub contribution practices
- Open-source fundamentals

Part XXXI — Career & Placement Preparation

115. Resume Preparation

- MERN Stack developer resume
- Full-stack developer resume
- ATS-friendly resume
- Technical skills presentation
- Project descriptions
- Quantifying project outcomes
- GitHub links
- Portfolio links

116. LinkedIn Profile Building

- Professional headline
- About section
- Skills
- Projects
- Certifications
- GitHub integration

- Portfolio integration
- Networking fundamentals

117. JavaScript Interview Preparation

- Scope
- Hoisting
- Closures
- `this`
- Prototypes
- Event loop
- Promises
- Async/await
- Arrays
- Objects
- ES6+
- Memory concepts
- Coding questions

118. React Interview Preparation

- Components
- JSX
- Props
- State
- Hooks
- `useEffect`
- Context
- Routing
- Forms
- Redux
- State management
- Performance optimization
- Re-rendering
- Controlled vs uncontrolled components
- Virtual DOM concepts

119. Node.js Interview Preparation

- Node architecture
- Event loop
- Non-blocking I/O
- Modules
- npm
- Streams
- Buffers
- Async programming
- Error handling
- Scalability
- Performance

120. Express.js Interview Preparation

- Routing
- Middleware

- Request/response
- Controllers
- REST APIs
- Validation
- Error handling
- Authentication
- Authorization
- CORS
- Security
- Project architecture

121. MongoDB Interview Preparation

- SQL vs NoSQL
- Documents
- Collections
- CRUD
- Aggregation
- Indexing
- Schema design
- Embedding vs referencing
- Transactions
- Query optimization
- Mongoose

122. Web & API Interview Preparation

- HTTP
- HTTPS
- REST
- HTTP methods
- Status codes
- Cookies
- Sessions
- JWT
- OAuth
- CORS
- CSRF
- XSS
- Authentication
- Authorization
- API security

123. System Design Fundamentals

- Three-tier architecture
- Client-server architecture
- Monolith
- Microservices
- REST APIs
- Database design
- MongoDB modeling
- Caching
- Redis

- Load balancing
- Horizontal scaling
- Vertical scaling
- Message queues
- CDN
- Authentication
- File storage
- High availability fundamentals

Practice systems:

- URL shortener
- E-commerce platform
- Chat application
- Job portal
- Learning Management System
- Notification service

124. Mock Interviews & Coding Practice

- JavaScript coding challenges
- React practical exercises
- Node.js coding challenges
- MongoDB queries
- REST API design exercises
- Debugging challenges
- Project explanation
- System design rounds
- Technical mock interviews
- HR mock interviews
- Resume reviews
- Interview communication practice

Recommended MERN Stack Technology Stack

- Layer — Technologies
- Programming — JavaScript, TypeScript
- Web Fundamentals — HTML5, CSS3
- UI Styling — Bootstrap / Tailwind CSS
- Frontend — React
- Frontend Routing — React Router
- State Management — Context API, Redux Toolkit / Zustand
- Server-State Management — TanStack Query concepts
- Backend Runtime — Node.js
- Backend Framework — Express.js
- Database — MongoDB
- ODM — Mongoose
- APIs — REST, JSON
- Authentication — JWT, OAuth/OIDC concepts
- API Validation — Zod / Joi / express-validator
- API Documentation — Swagger / OpenAPI
- API Testing — Postman
- Real-Time — WebSockets / Socket.IO
- Caching — Redis

- Background Jobs — BullMQ concepts
- Testing — Jest / Vitest, React Testing Library, Supertest
- Version Control — Git, GitHub
- Containers — Docker, Docker Compose
- CI/CD — GitHub Actions
- Cloud — AWS / Azure / GCP fundamentals
- Database Cloud — MongoDB Atlas
- Monitoring — Logging, health checks, observability fundamentals
- AI-Assisted Development — GitHub Copilot, ChatGPT, Claude/Cursor
- Generative AI — LLM APIs, embeddings, RAG fundamentals
- Project Management — Agile, Scrum, Jira

Recommended Learning Sequence

Web Fundamentals → HTML → CSS → JavaScript → Advanced JavaScript → TypeScript → Git/GitHub → React → React Router → State Management → Node.js → Express.js → MongoDB → Mongoose → REST API Development → Authentication & Security → React + Backend Integration → Testing → Redis → Real-Time Applications → Docker → CI/CD → Cloud Deployment → Microservices Fundamentals → AI-Assisted Development → Generative AI Integration → Capstone Project → Interview Preparation

Suggested Course Positioning

For Archer Infotech, I would position this course as:

AI-Assisted MERN Stack Full-Stack Development Course

with the core stack:

MongoDB + Express.js + React + Node.js + JavaScript/TypeScript + REST APIs + Docker + Cloud + Generative AI

The core architecture students should understand is:

React UI → React Services/Hooks → Node.js/Express REST API → Mongoose → MongoDB

and at an advanced level:

React → Node.js/Express → MongoDB + Redis + AI/LLM Services → Docker → CI/CD → Cloud

This positioning makes the course stronger than a traditional MERN syllabus because students learn not only **MongoDB, Express, React, and Node.js**, but also **TypeScript, application architecture, authentication, testing, real-time development, DevOps, cloud deployment, AI-assisted coding, and Generative AI integration**.

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.